

From: <https://chatgpt.com/c/6a8b0932-b518-83ee-b51e-511380bd96fa>

you asked: go through

message time: 2026-08-23 20:20:26

go through the previous chats from start to end completely which are attached files and responses how detailed- <https://chatgpt.com/c/6a7f33eb-07f4-83ee-8229-383f952dcf93> , <https://chatgpt.com/c/6a8a1a24-1218-83e9-9f55-db7ec639b9a5> , <https://chatgpt.com/c/6a8a20f0-f3fc-83ee-891b-ba811975393f> , <https://chatgpt.com/c/6a8a2356-f96c-83ee-950e-82f71a5e00eb> , <https://chatgpt.com/c/6a8a251a-31f8-83e8-9107-c2dfbcaab421> , <https://chatgpt.com/c/6a8a279f-3c40-83ee-8aa1-24574734f5ea> , <https://chatgpt.com/c/6a8a2959-22ac-83ee-b837-2220612a953b>

And continue further according to previous - Module 1,2,3,4,5,6,7,8,9,10 completed as above their links given and also attached files . Now continue with Module 11: 100 DSA problems specifically for 2-year Java SDE interviews.

For your goal, I recommend turning this into a structured “2-Year Java + Spring Boot Product Company Interview Bible” with: Module 1: 150 Core Java questions + answers Module 2: 120 Spring/Spring Boot questions + internal working Module 3: 80 JPA/Hibernate questions Module 4: 75 REST/API questions Module 5: 75 Microservices questions Module 6: 75 Spring Security/JWT questions Module 7: 50 SQL questions + queries Module 8: 75 Multithreading questions Module 9: 50 Kafka + Redis questions Module 10: 50 Docker/AWS questions Module 11: 100 DSA problems specifically for 2-year Java SDE interviews Module 12: 50 LLD/system-design scenarios Module 13: 100 project/resume follow-up questions Module 14: Company-specific patterns for Amazon, Google, Microsoft, Adobe, Uber, Walmart, Mastercard, Morgan Stanley, Cisco, Salesforce, Atlassian, Oracle, etc.

Most importantly, for every question I'd structure it as: Question → Short interview answer → Detailed explanation → Real project example → Follow-up question → Tricky interviewer counter-question → Best answer → Code/diagram where relevant That format will train you to answer follow-up questions instead of just memorizing definitions.

spring boot interviews latest for 2 year experience with questions and answers asked in the MANGOS companies so on that questions based i can easily crack the interview rounds in top product based companies or MANGOS companies, give me all the questions with answers after going through websites, online published questions or any other references of videos or youtubes then give all in proper format and explanation of concepts. X10THINK

Absolutely. treated “MANGOS” as FAANG/MANGA-style top product companies and built this around what a 2-year Java + Spring Boot backend engineer is realistically expected to handle.

checked recent interview reports and published question banks rather than giving you a dsa questions list. Recent reports for **Mastercard, Target, Cisco, Morgan Stanley, Apptio, Infosys, Accenture, HCL and others** repeatedly show the same pattern: **Core Java + Spring/Spring Boot + REST +**

JPA/Hibernate + SQL + multithreading + microservices + project deep-dive + DSA, with product companies adding more scenario/LLD depth.

give Module 11: 100 DSA problems specifically for 2-year Java SDE interviews with questions with answer should be expected by interviewer on real interview scenario based discussion also. X10THINK

chatgpt response: I'll conti

I'll continue from the established interview-bible structure and make Module 11 scenario-driven: each problem will include the expected approach, Java solution, complexity, and the follow-ups an interviewer is likely to ask. I'll also cross-check current interview patterns so the set is aimed at real 2-year backend interviews rather than generic DSA practice.

Module 11 — 100 DSA Problems for 2-Year Java SDE Interviews

We are continuing the same **2-Year Java + Spring Boot Product Company Interview Bible**. The previous material established the same progression: questions should move beyond definitions into **“how would you solve this?”**, **“why this approach?”**, **complexity**, **production relevance**, and **interviewer follow-ups**.

One limitation is worth stating clearly: the ChatGPT URLs you provided resolve here only to the public login shell, so I cannot read the private conversation bodies from those URLs. ([ChatGPT](#)) I *can* access the interview-bible files from your earlier work, and they confirm the intended structure and progression.

For the DSA module, I have also checked current 2026 material. Current interview guidance and recent candidate reports continue to emphasize **data structures**, **algorithms**, **efficient problem solving**, **coding correctness**, **edge cases**, and **explaining trade-offs**, while 2-year Java/backend interviews commonly combine DSA with project/backend discussion. ([Amazon.jobs](#))

Amazon's current SDE guidance, for example, explicitly evaluates syntactically correct code, efficiency, robustness, edge cases, maintainability and problem solving rather than memorization. ([Amazon.jobs](#))

How to use these 100 problems

For every problem, train yourself to speak in this order:

Clarify → brute force → identify pattern → optimized approach → code → dry run → time complexity → space complexity → edge cases → interviewer follow-up → production connection

That is much more valuable for a 2-year candidate than memorizing 100 LeetCode solutions.

SECTION A — ARRAYS + HASHING

Q1–20

Q1. Two Sum

Interview question

Given an integer array and target, return indices of two numbers whose sum equals the target.

Short interview answer

Use a `HashMap<Integer, Integer>` storing `value -> index`. For every element, check whether `target - current` already exists.

Detailed explanation

Brute force checks every pair:

```
text
O(n2)
```

Hashing reduces the lookup to approximately `O(1)` per element.

Example:

```
text
nums = [2,7,11,15]
target = 9

2 → store
7 → 9 - 7 = 2 → found
```

Java

```
java
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();

    for (int i = 0; i < nums.length; i++) {
        int needed = target - nums[i];

        if (map.containsKey(needed)) {
            return new int[]{map.get(needed), i};
        }

        map.put(nums[i], i);
    }
}
```

```
return new int[0];  
}
```

Complexity

```
text  
  
Time: O(n)  
Space: O(n)
```

Real backend example

Finding two transaction amounts that combine to a reconciliation target.

Follow-up

What if the interviewer asks for the **values instead of indexes**?

Return:

```
java  
return new int[]{needed, nums[i]};
```

Tricky counter-question

Why not use `HashSet`?

Because we need the **index**, not only existence.

Q2. Contains Duplicate

Question

Determine whether an array contains duplicates.

Best answer

Use a `HashSet`.

```
java  
public boolean containsDuplicate(int[] nums) {  
    Set<Integer> seen = new HashSet<>();  
  
    for (int n : nums) {  
        if (!seen.add(n)) {  
            return true;  
        }  
    }  
}
```

```
    }  
  
    return false;  
}
```

Complexity

```
text  
Time: O(n)  
Space: O(n)
```

Follow-up

Can you solve without extra space?

Yes, sort first:

```
text  
sort → O(n log n)  
scan → O(n)
```

Total:

```
text  
O(n log n)
```

Tricky question

Why does `Set.add()` help?

`add()` returns `false` if the element already exists.

Production example

Detecting duplicate request IDs before processing a batch.

Q3. Valid Anagram

Question

Determine whether two strings contain the same characters with the same frequencies.

Approach

Count characters.

```

java
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) {
        return false;
    }

    int[] count = new int[26];

    for (char c : s.toCharArray()) {
        count[c - 'a']++;
    }

    for (char c : t.toCharArray()) {
        count[c - 'a']--;
    }

    for (int x : count) {
        if (x != 0) {
            return false;
        }
    }

    return true;
}

```

Complexity

text

$O(n)$

Follow-up

What if Unicode is allowed?

Use:

```

java
Map<Character, Integer>

```

instead of fixed array.

Production example

Case-insensitive token normalization or duplicate-content detection.

Q4. Group Anagrams

Question

Group strings that are anagrams.

Example:

```
text
eat
tea
ate
bat

=> [[eat,tea,ate],[bat]]
```

Approach

Create a canonical key.

Option 1:

```
text
sort characters
```

Example:

```
text
eat → aet
tea → aet
ate → aet
```

Java

```
java
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();

    for (String s : strs) {
        char[] chars = s.toCharArray();
        Arrays.sort(chars);
```

```

        String key = new String(chars);

        map.computeIfAbsent(key, k -> new ArrayList<>())
            .add(s);
    }

    return new ArrayList<>(map.values());
}

```

Complexity

text

$O(n * k \log k)$

where k is average string length.

Follow-up

Can you do better?

Yes, frequency-based keys can reduce character sorting cost.

Production example

Grouping equivalent product-search keywords.

Q5. Top K Frequent Elements

Question

Return the k most frequent elements.

Approach

1. Count using `HashMap`.
2. Maintain a min-heap of size k .

```

java
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();

    for (int n : nums) {
        freq.merge(n, 1, Integer::sum);
    }
}

```



```

PriorityQueue<int[]> pq =
    new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));

for (var entry : freq.entrySet()) {
    pq.offer(new int[]{entry.getKey(), entry.getValue()});

    if (pq.size() > k) {
        pq.poll();
    }
}

int[] result = new int[k];

for (int i = k - 1; i >= 0; i--) {
    result[i] = pq.poll()[0];
}

return result;
}

```

Complexity

text

$O(n \log k)$

Follow-up

Why min-heap rather than max-heap?

Because we want to efficiently remove the **least frequent among the current top K**.

Backend example

Top-K most frequently requested products/API endpoints.

Q6. Product of Array Except Self

Question

Return an array where:

text

`answer[i] = product of every element except nums[i]`

without division.

Key idea

Prefix $\times$ suffix.

text

```
nums = [1,2,3,4]
```

prefix:

```
[1,1,2,6]
```

suffix:

```
[24,12,4,1]
```

Java

java

```
public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];

    result[0] = 1;

    for (int i = 1; i < n; i++) {
        result[i] = result[i - 1] * nums[i - 1];
    }

    int suffix = 1;

    for (int i = n - 1; i >= 0; i--) {
        result[i] *= suffix;
        suffix *= nums[i];
    }

    return result;
}
```

Complexity

text

Time: $O(n)$

Space: $O(1)$ extra

Tricky question

Why not division?

Zero makes division-based solutions incorrect and the problem explicitly disallows it.

Q7. Best Time to Buy and Sell Stock

Question

Find the maximum profit from one transaction.

Core idea

Maintain minimum price seen so far.

java

```
public int maxProfit(int[] prices) {  
    int minPrice = Integer.MAX_VALUE;  
    int maxProfit = 0;  
  
    for (int price : prices) {  
        minPrice = Math.min(minPrice, price);  
        maxProfit = Math.max(maxProfit, price - minPrice);  
    }  
  
    return maxProfit;  
}
```

Complexity

text

$O(n)$

Interview explanation

At day i :

text

profit = todayPrice - cheapestPreviousPrice

Follow-up

What if multiple transactions are allowed?

That becomes a different problem:

```
text
sum every positive difference
```

Q8. Maximum Subarray

Question

Find the contiguous subarray with maximum sum.

Pattern

Kadane's Algorithm

```
java
public int maxSubArray(int[] nums) {
    int current = nums[0];
    int best = nums[0];

    for (int i = 1; i < nums.length; i++) {
        current = Math.max(nums[i], current + nums[i]);
        best = Math.max(best, current);
    }

    return best;
}
```

Complexity

```
text
Time: O(n)
Space: O(1)
```

Interview insight

At each position:

```
text
Should I:
start a new subarray?
```

OR

extend the old one?

Follow-up

How do you return the actual subarray?

Track:

```
text
start
bestStart
bestEnd
```

Q9. Majority Element

Question

Find the element appearing more than $n/2$ times.

Best approach

Boyer-Moore Voting Algorithm

```
java
public int majorityElement(int[] nums) {
    int candidate = 0;
    int count = 0;

    for (int n : nums) {
        if (count == 0) {
            candidate = n;
        }

        count += (n == candidate) ? 1 : -1;
    }

    return candidate;
}
```

Why it works

Every different number cancels one vote of the candidate.

Because the majority is greater than $n/2$, it survives.

Complexity

```
text
O(n) time
O(1) space
```

Tricky question

What if a majority is **not guaranteed**?

Verify the candidate with a second pass.

Q10. Missing Number

Question

Array contains numbers from $0..n$ with one missing.

Approach 1

Use sum.

```
text
expected = n(n+1)/2
```

Potential integer overflow.

Better interview trick

XOR.

```
java
public int missingNumber(int[] nums) {
    int xor = nums.length;

    for (int i = 0; i < nums.length; i++) {
        xor ^= i;
        xor ^= nums[i];
    }

    return xor;
}
```

Complexity

text

$O(n)$

$O(1)$

Follow-up

Why XOR?

text

$x \oplus x = 0$

$x \oplus 0 = x$

All existing values cancel.

Q11. Longest Consecutive Sequence

Question

Find longest consecutive sequence in an unsorted array.

Approach

Put everything into a `HashSet`.

Only begin counting when:

text

$n - 1$ does not exist

Java

java

```
public int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();

    for (int n : nums) {
        set.add(n);
    }

    int best = 0;

    for (int n : set) {
        if (!set.contains(n - 1)) {
            int current = n;
```

```

        int length = 1;

        while (set.contains(current + 1)) {
            current++;
            length++;
        }

        best = Math.max(best, length);
    }
}

return best;
}

```

Complexity

Average:

```

text
O(n)

```

Interview trap

Don't start a sequence from every number. That can become inefficient.

Q12. Subarray Sum Equals K

Question

Count subarrays whose sum equals `k`.

Key insight

Prefix sums.

If:

```

text
prefix[j] - prefix[i] = k

```

then:

```

text
prefix[i] = prefix[j] - k

```


Java

```
java
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> map = new HashMap<>();
    map.put(0, 1);

    int prefix = 0;
    int count = 0;

    for (int n : nums) {
        prefix += n;

        count += map.getOrDefault(prefix - k, 0);

        map.merge(prefix, 1, Integer::sum);
    }

    return count;
}
```

Complexity

```
text
O(n)
```

Tricky follow-up

Why initialize:

```
java
map.put(0, 1);
```

Because a prefix itself may equal `k`.

Q13. Merge Intervals

Question

Merge overlapping intervals.

Example:

text

[1,3] [2,6] [8,10]

=> [1,6] [8,10]

Approach

Sort by start time, then merge.

java

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));

    List<int[]> result = new ArrayList<>();

    for (int[] current : intervals) {
        if (result.isEmpty() ||
            result.get(result.size() - 1)[1] < current[0]) {

            result.add(current);
        } else {
            result.get(result.size() - 1)[1] =
                Math.max(result.get(result.size() - 1)[1], current[1]);
        }
    }

    return result.toArray(new int[0][]);
}
```

Complexity

text

$O(n \log n)$

Real-world example

Merge overlapping:

text

employee schedules
maintenance windows
booking ranges

Q14. Insert Interval

Core approach

Process three phases:

```
text
1. intervals completely before new interval
2. overlapping intervals
3. intervals after it
```

Complexity

```
text
O(n)
```

when existing intervals are already sorted.

Follow-up

Why isn't sorting necessary?

Because input is already ordered.

Q15. Rotate Array

Question

Rotate array to the right by k .

Best method

Three reversals.

```
text
reverse entire
reverse first k
reverse remaining
```

Java

```
java
public void rotate(int[] nums, int k) {
    k %= nums.length;

    reverse(nums, 0, nums.length - 1);
```

```

        reverse(nums, 0, k - 1);
        reverse(nums, k, nums.length - 1);
    }

    private void reverse(int[] a, int l, int r) {
        while (l < r) {
            int temp = a[l];
            a[l++] = a[r];
            a[r--] = temp;
        }
    }
}

```

Complexity

text

$O(n)$

$O(1)$

Tricky question

Why:

java

```
k %= nums.length;
```

Because rotating by n , $2n$, etc. produces the original array.

Q16. Sort Colors

Question

Sort:

text

0, 1, 2

in one pass.

Approach

Dutch National Flag.

Maintain:

text

low
mid
high

Java

java

```
public void sortColors(int[] nums) {  
    int low = 0;  
    int mid = 0;  
    int high = nums.length - 1;  
  
    while (mid <= high) {  
        if (nums[mid] == 0) {  
            swap(nums, low++, mid++);  
        } else if (nums[mid] == 1) {  
            mid++;  
        } else {  
            swap(nums, mid, high--);  
        }  
    }  
}
```

Complexity

text

$O(n)$
 $O(1)$

Follow-up

Why don't we increment `mid` after swapping with `high`?

Because the element brought from `high` hasn't been examined.

Q17. Find the Duplicate Number

Important pattern

Treat array indexes/values as a linked list.

Use:

Floyd Cycle Detection

Why?

If:

```
text
nums[i] -> nums[nums[i]]
```

we effectively create a cycle.

Complexity

```
text
O(n)
O(1)
```

Interview follow-up

Why can't you simply use a `HashSet`?

You can, but it uses:

```
text
O(n)
```

extra space.

The cycle solution achieves constant extra space.

Q18. Set Matrix Zeroes

Question

If a matrix cell is zero, set its entire row and column to zero.

Tricky requirement

Use $O(1)$ extra space.

Technique

Use the first row and first column as markers.

Interview focus

Interviewers care more about understanding the marker technique than memorizing code.

Complexity

```
text
```

```
O(m*n)
```

```
O(1)
```

Follow-up

What extra state is needed?

A boolean indicating whether the **first column originally contained zero**, because it is being reused as metadata.

Q19. Spiral Matrix

Approach

Maintain four boundaries:

```
text
```

```
top
```

```
bottom
```

```
left
```

```
right
```

Process:

```
text
```

```
left → right
```

```
top → bottom
```

```
right → left
```

```
bottom → top
```

Complexity

```
text
```

```
O(m*n)
```

Tricky interviewer question

Why check boundaries after each directional traversal?

Because for odd-sized matrices, a row/column can become exhausted before its reverse traversal.

Q20. Search in a 2D Matrix

If every row is sorted and the first element of each row is greater than the previous row's last element, treat the matrix as one sorted array.

Mapping

For index `mid`:

```
text
row = mid / columns
col = mid % columns
```

Complexity

```
text
O(log(m*n))
```

Interview follow-up

What if rows and columns are independently sorted but the row boundary condition doesn't hold?

Use a different strategy, such as top-right search.

SECTION B — SLIDING WINDOW + TWO POINTERS

Q21–35

Q21. Best Time to Buy and Sell Stock II

Multiple transactions allowed.

Key insight

Capture every positive upward movement.

```
java
profit += Math.max(0, prices[i] - prices[i - 1]);
```

Complexity

```
text
O(n)
```

Tricky question

Why is this valid?

Every rising sequence's total increase is equal to the sum of its day-to-day increases.

Q22. Longest Substring Without Repeating Characters

This is particularly important for Java backend interviews; recent reports for Java backend roles have included this problem directly. ([CodingKaro](#))

Approach

Sliding window.

```
java
public int lengthOfLongestSubstring(String s) {
    Set<Character> set = new HashSet<>();

    int left = 0;
    int best = 0;

    for (int right = 0; right < s.length(); right++) {
        while (set.contains(s.charAt(right))) {
            set.remove(s.charAt(left++));
        }

        set.add(s.charAt(right));

        best = Math.max(best, right - left + 1);
    }

    return best;
}
```

Complexity

```
text
O(n)
```

Better version

Use an index map to jump `left`.

Tricky question

Why is the sliding window not $O(n^2)$?

Because every element enters and leaves the window at most a constant number of times.

Q23. Longest Repeating Character Replacement

Pattern

Sliding window + frequency.

Condition:

```
text
windowSize - maxFrequency <= k
```

means all other characters can be replaced.

Complexity

```
text
O(n)
```

Follow-up

Why don't we recompute `maxFrequency` every time?

Because keeping the largest historical frequency is sufficient for the correctness of the window-expansion strategy.

Q24. Permutation in String

Question

Determine whether `s2` contains a permutation of `s1`.

Approach

Fixed-size sliding window.

Compare frequency arrays.

Complexity

For a fixed alphabet:

```
text
O(n)
```

Tricky question

Why is the window exactly:

text

```
s1.length()
```

Because any permutation of `s1` has exactly the same size.

Q25. Minimum Size Subarray Sum

Approach

Variable sliding window for positive integers.

text

```
expand right
while sum >= target:
    update answer
    shrink left
```

Complexity

text

```
 $O(n)$ 
```

Important condition

Sliding window works cleanly because all numbers are positive.

Tricky question

What if negative values are allowed?

The standard sliding-window solution no longer works; use prefix sums + monotonic deque techniques depending on the exact problem.

Q26. Maximum Sum Subarray of Size K

Pattern

Fixed sliding window.

java

```
public int maxSum(int[] nums, int k) {
    int sum = 0;

    for (int i = 0; i < k; i++) {
        sum += nums[i];
    }
}
```

```

    }

    int best = sum;

    for (int i = k; i < nums.length; i++) {
        sum += nums[i];
        sum -= nums[i - k];

        best = Math.max(best, sum);
    }

    return best;
}

```

Complexity

text
 $O(n)$

Q27. Minimum Window Substring

Classic hard/intermediate sliding-window problem.

Pattern

text
 required frequency
 current frequency
 formed count
 left/right pointers

When the current window contains all required characters, shrink from the left.

Complexity

text
 $O(n)$

Interview expectation

You should be able to explain:

text

```
why expand
why shrink
when window becomes valid
when answer is updated
```

not merely reproduce code.

Q28. Maximum Number of Vowels in a Substring of Length K

Fixed window.

```
java
public int maxVowels(String s, int k) {
    int count = 0;

    for (int i = 0; i < k; i++) {
        if (isVowel(s.charAt(i))) {
            count++;
        }
    }

    int best = count;

    for (int i = k; i < s.length(); i++) {
        if (isVowel(s.charAt(i))) count++;
        if (isVowel(s.charAt(i - k))) count--;

        best = Math.max(best, count);
    }

    return best;
}
```

Q29. Move Zeroes

Move all zeroes to end while preserving relative order.

Best approach

Two pointers.

```

java

public void moveZeroes(int[] nums) {
    int insert = 0;

    for (int n : nums) {
        if (n != 0) {
            nums[insert++] = n;
        }
    }

    while (insert < nums.length) {
        nums[insert++] = 0;
    }
}

```

Complexity

```

text

O(n)
O(1)

```

Q30. Container With Most Water

Pattern

Two pointers.

Start:

```

text

left = 0
right = n-1

```

Area:

```

text

min(height[left], height[right]) * (right-left)

```

Move the shorter side.

Tricky question

Why not move the taller side?

Because width decreases, and the taller side cannot compensate for the limiting shorter height.

Q31. 3Sum

Approach

1. Sort.
2. Fix one number.
3. Solve remaining two-sum using two pointers.

Complexity

```
text
O(n2)
```

Important interview point

Avoid duplicate triplets by skipping equal values.

```
java
if (i > 0 && nums[i] == nums[i - 1]) continue;
```

Q32. 4Sum

Extension of 3Sum.

Approach

Fix two elements then two-pointer the remaining range.

Complexity

```
text
O(n3)
```

Follow-up

How would you improve if `k` changes dynamically?

General `k-Sum` recursively reduces:

```
text
k-Sum → (k-1)-Sum → ... → 2-Sum
```

Q33. Trapping Rain Water

Approach

Two-pointer version.

Maintain:

```
text
leftMax
rightMax
```

Move the side whose maximum boundary is smaller.

Complexity

```
text
O(n)
O(1)
```

Production analogy

Can help test whether the candidate recognizes **boundary-driven aggregation**, useful beyond this exact problem.

Q34. Remove Duplicates from Sorted Array

Use two pointers:

```
text
read
write
```

Complexity

```
text
O(n)
O(1)
```

Key observation

Because the array is sorted, duplicate values are adjacent.

Q35. Remove Element

Overwrite elements not matching the target.


```

java

public int removeElement(int[] nums, int val) {
    int write = 0;

    for (int n : nums) {
        if (n != val) {
            nums[write++] = n;
        }
    }

    return write;
}

```

SECTION C — LINKED LISTS

Q36–45

Q36. Reverse a Linked List

Question

Reverse:

```

text

1 → 2 → 3 → null

```

into:

```

text

3 → 2 → 1 → null

```

Java

```

java

public ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode current = head;

    while (current != null) {
        ListNode next = current.next;
        current.next = prev;
    }
}

```

```

        prev = current;
        current = next;
    }

    return prev;
}

```

Complexity

text

$O(n)$

$O(1)$

Tricky question

Why save `next` before changing `current.next`?

Otherwise the remaining list is lost.

Q37. Detect Cycle in Linked List

Use Floyd's algorithm.

text

`slow += 1`

`fast += 2`

If they meet, there is a cycle.

Complexity

text

$O(n)$

$O(1)$

Follow-up

Can you find the cycle entrance?

Yes. After the first meeting, move one pointer to head. Then move both one step at a time.

Q38. Middle of Linked List

Fast and slow pointers.

java

```
public ListNode middleNode(ListNode head) {  
    ListNode slow = head;  
    ListNode fast = head;  
  
    while (fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
    }  
  
    return slow;  
}
```

Complexity

text

$O(n)$
 $O(1)$

Q39. Merge Two Sorted Lists

Use a dummy node.

java

```
public ListNode mergeTwoLists(ListNode a, ListNode b) {  
    ListNode dummy = new ListNode(0);  
    ListNode current = dummy;  
  
    while (a != null && b != null) {  
        if (a.val <= b.val) {  
            current.next = a;  
            a = a.next;  
        } else {  
            current.next = b;  
            b = b.next;  
        }  
  
        current = current.next;  
    }  
}
```

```
current.next = (a != null) ? a : b;

return dummy.next;
}
```

Complexity

```
text
O(n + m)
```

Q40. Remove Nth Node From End

Two pointers separated by n .

Pattern

```
text
fast moves n steps
slow begins
move both
```

Use a dummy node to simplify removal of the head.

Complexity

```
text
O(n)
O(1)
```

Q41. Palindrome Linked List

Best approach

1. Find middle.
2. Reverse second half.
3. Compare halves.

Complexity

```
text
O(n)
O(1)
```

Follow-up

Should we restore the list?

In production code, restoring it is often preferable if callers expect the input structure to remain unchanged.

Q42. Intersection of Two Linked Lists

Elegant solution

When pointer **A** reaches null:

```
text
A = headB
```

When pointer **B** reaches null:

```
text
B = headA
```

They synchronize path lengths.

Complexity

```
text
O(n + m)
O(1)
```

Tricky question

Intersection means the same **node object**, not just equal values.

Q43. Add Two Numbers

Each linked list represents a number in reverse order.

Maintain:

```
text
carry
```

Formula

```
text
sum = a + b + carry
digit = sum % 10
carry = sum / 10
```

Complexity

```
text
O(max(n,m))
```

Q44. Reorder List

Transform:

```
text
1 → 2 → 3 → 4 → 5
```

into:

```
text
1 → 5 → 2 → 4 → 3
```

Steps

```
text
find middle
reverse second half
merge two halves
```

This combines several patterns and is a very good interview problem.

Q45. Copy List With Random Pointer

Each node has:

```
text
next
random
```

Important approaches

Approach 1: HashMap old node → new node.

```
text
O(n) time
O(n) space
```

Approach 2: Interweave copied nodes.

text

$O(n)$

$O(1)$ extra

Interview follow-up

Why is this harder than copying a normal list?

Because `random` may point to any node, including a node not yet processed.

SECTION D — STACK + QUEUE

Q46–55

Q46. Valid Parentheses

Use a stack.

java

```
public boolean isValid(String s) {  
    Deque<Character> stack = new ArrayDeque<>();  
  
    for (char c : s.toCharArray()) {  
        if (c == '(') stack.push('(');  
        else if (c == '[') stack.push('[');  
        else if (c == '{') stack.push('{');  
        else if (stack.isEmpty() || stack.pop() != c) return false;  
    }  
  
    return stack.isEmpty();  
}
```

Complexity

text

$O(n)$

Interview insight

Stack naturally models **nested state**.

Q47. Min Stack

Support:

```
text
push
pop
top
getMin
```

all in $O(1)$.

Approach

Maintain two stacks:

```
text
main stack
minimum stack
```

Tricky question

Can one stack do it?

Yes, with encoded values, but two stacks are usually clearer.

Q48. Implement Queue Using Stacks

Use:

```
text
inStack
outStack
```

When `outStack` is empty, transfer all elements from `inStack`.

Amortized complexity

```
text
 $O(1)$ 
```

Tricky question

Why is it not $O(n)$ every time?

Each element transfers from one stack to the other at most once per direction across operations, yielding amortized constant time.

Q49. Daily Temperatures

Pattern

Monotonic decreasing stack of indexes.

For each temperature:

```
text
while current > temperature at stack top
```

resolve that older day.

Complexity

```
text
O(n)
```

Important insight

A monotonic stack is useful whenever you need:

```
text
next greater
next smaller
previous greater
previous smaller
```

Q50. Next Greater Element

Use a monotonic decreasing stack.

Complexity

```
text
O(n)
```

Follow-up

Why store indices rather than values?

Because we often need:

```
text
distance
```

```
position
answer[index]
```

Q51. Largest Rectangle in Histogram

Key idea

Monotonic increasing stack.

When the current bar is smaller, repeatedly calculate rectangles for bars that can no longer expand.

Complexity

```
text
O(n)
```

Tricky question

Why append a sentinel zero-height bar?

It forces remaining stack elements to be processed.

Q52. Sliding Window Maximum

Use a **monotonic deque**.

The deque stores indexes in decreasing order of values.

Complexity

```
text
O(n)
```

Why deque?

The front always gives the maximum of the current window.

Q53. Evaluate Reverse Polish Notation

Use a stack.

Example:

```
text
2 1 + 3 *
```

means:

```
text
(2 + 1) * 3
```

Complexity

```
text
O(n)
```

Tricky point

For:

```
text
a - b
a / b
```

operand order matters.

Q54. Decode String

Example:

```
text
3[a2[c]]
```

Pattern

Use stacks for:

```
text
repeat counts
strings
```

When `]` appears, expand the nested expression.

Complexity

Depends on output size.

Potentially larger than input because decoding itself may generate a large result.

Q55. Design a Stack With O(1) Min/Max

This is a common extension question.

Maintain:

```
text
value
currentMinimum
currentMaximum
```

for each stack node.

That makes:

```
text
getMin()
getMax()
```

constant time.

SECTION E — BINARY SEARCH

Q56–65

Q56. Binary Search

Java

```
java
public int binarySearch(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) return mid;

        if (nums[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }

    return -1;
}
```

Complexity

```
text
O(log n)
```

Tricky question

Why:

```
java
left + (right-left)/2
```

instead of:

```
java
(left+right)/2
```

To avoid integer overflow.

Q57. First and Last Position of Element

Perform binary search twice:

```
text
first occurrence
last occurrence
```

Complexity

```
text
O(log n)
```

Q58. Search in Rotated Sorted Array

Example:

```
text
[4,5,6,7,0,1,2]
```

At least one half is always sorted.

Determine whether target lies in that sorted half.

Complexity

```
text
O(log n)
```

Follow-up

What if duplicates exist?

The logic becomes more complicated because:

```
text
nums[left] == nums[mid] == nums[right]
```

may prevent identifying which half is sorted.

Q59. Find Minimum in Rotated Sorted Array

Compare middle with right boundary.

```
text
if nums[mid] > nums[right]
    left = mid + 1
else
    right = mid
```

Complexity

```
text
O(log n)
```

Q60. Peak Element

A peak is an element greater than neighbors.

Compare:

```
text
nums[mid]
nums[mid+1]
```

If increasing, peak must exist on the right.

If decreasing, a peak exists on left/self.

Complexity

```
text
O(log n)
```

Q61. Search a 2D Matrix

Already discussed earlier, but this is an important binary-search pattern.

Flatten logically:

```
text
mid
row = mid / columns
col = mid % columns
```

Complexity

```
text
O(log(m*n))
```

Q62. Koko Eating Bananas

Important pattern

Binary search on answer.

Search:

```
text
speed = 1 ... max(piles)
```

For each speed calculate required hours.

Complexity

```
text
O(n log(maxPile))
```

Interview insight

Binary search is not limited to sorted arrays.

Ask:

Is there a monotonic feasibility condition?

If yes, binary-search-on-answer may be possible.

Q63. Capacity to Ship Packages Within D Days

Again:

binary search on answer.

Search minimum feasible ship capacity.

Feasibility function

```
text
canShip(capacity)
```

returns true/false.

Complexity

```
text
O(n log(sum(weights)))
```

Q64. Find Minimum in Rotated Array II

Same idea as Q59 but duplicates create ambiguity.

When:

```
text
nums[mid] == nums[right]
```

you may need:

```
text
right--
```

Complexity

Average/typical:

```
text
O(log n)
```

Worst case can degrade to:

```
text
O(n)
```


Q65. Median of Two Sorted Arrays

This is one of the more difficult interview questions.

Core technique

Binary search the partition of the smaller array.

Find a partition where:

```
text
leftA <= rightB
leftB <= rightA
```

Then calculate median from the partition boundaries.

Complexity

```
text
O(log(min(m,n)))
```

Interview expectation

For a 2-year candidate, explaining the partition idea clearly is often more important than instantly memorizing the implementation.

SECTION F — TREES + BST

Q66–75

Q66. Maximum Depth of Binary Tree

Recursive solution

```
java
public int maxDepth(TreeNode root) {
    if (root == null) return 0;

    return 1 + Math.max(
        maxDepth(root.left),
        maxDepth(root.right)
    );
}
```

Complexity

text

$O(n)$

Tricky question

What happens for a skewed tree?

Recursion depth becomes $O(n)$.

Q67. Invert Binary Tree

Swap left/right recursively.

java

```
public TreeNode invertTree(TreeNode root) {  
    if (root == null) return null;  
  
    TreeNode temp = root.left;  
    root.left = root.right;  
    root.right = temp;  
  
    invertTree(root.left);  
    invertTree(root.right);  
  
    return root;  
}
```

Q68. Validate Binary Search Tree

Critical point

Don't simply check:

text

$left < node < right$

for immediate children.

Use a valid range.

java

```
public boolean isValidBST(TreeNode root) {  
    return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);  
}
```

```

}

private boolean validate(TreeNode node, long min, long max) {
    if (node == null) return true;

    if (node.val <= min || node.val >= max) {
        return false;
    }

    return validate(node.left, min, node.val)
        && validate(node.right, node.val, max);
}

```

Complexity

text

$O(n)$

Q69. Binary Tree Level Order Traversal

Use BFS with a queue.

```

java
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();

    if (root == null) return result;

    Queue<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();

        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();

            level.add(node.val);

```

```

        if (node.left != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }

    result.add(level);
}

return result;
}

```

Complexity

text

$O(n)$

Q70. Lowest Common Ancestor of BST

Because BST is ordered:

text

```

node < p,q → go right
node > p,q → go left
otherwise node is LCA

```

Complexity

text

$O(h)$

Q71. Lowest Common Ancestor of Binary Tree

No BST ordering can be assumed.

Recursive approach:

text

```

if node == null/p/q → return node
left = search(left)
right = search(right)

if left != null && right != null → node is LCA

```

Complexity

```
text
O(n)
```

Q72. Diameter of Binary Tree

Key insight

For each node:

```
text
leftHeight + rightHeight
```

may form the best diameter passing through that node.

Complexity

```
text
O(n)
```

Tricky question

Why not independently calculate heights repeatedly?

That can produce $O(n^2)$ on skewed trees.

Compute height and diameter together in one DFS.

Q73. Balanced Binary Tree

A tree is balanced if height difference between left/right subtree is at most one.

Optimized solution

Return:

```
text
height
```

or:

```
text
-1 = unbalanced
```

Complexity

```
text
O(n)
```

Q74. Kth Smallest Element in BST

In-order traversal of a BST produces sorted order.

```
text
left
node
right
```

So the k th visited node is the answer.

Complexity

```
text
O(h + k)
```

depending on implementation and tree shape.

Q75. Serialize and Deserialize Binary Tree

Interview purpose

Tests whether you can convert a recursive data structure into a transferable representation.

Typical solution:

```
text
preorder + null markers
```

Example:

```
text
1,2,null,null,3,4,null,null,5
```

Backend connection

This resembles serialization/deserialization concepts encountered in:

```
text
JSON
caches
```

```
messaging
distributed systems
```

SECTION G — HEAP + GREEDY

Q76–82

Q76. Kth Largest Element in Array

Use a min-heap of size k .

```
text
smallest of top k stays at heap root
```

Complexity

```
text
 $O(n \log k)$ 
```

Alternative

Quickselect average:

```
text
 $O(n)$ 
```

but worst-case:

```
text
 $O(n^2)$ 
```

Q77. Merge K Sorted Lists

Use a min-heap containing the head node from each list.

Each time:

```
text
poll smallest
add its next
```

Complexity

```
text
O(N log k)
```

where:

```
text
N = total elements
k = number of lists
```

Real-world example

Merging sorted result streams from multiple shards.

Q78. Find Median From Data Stream

Maintain two heaps:

```
text
maxHeap → lower half
minHeap → upper half
```

Balance their sizes.

Median:

```
text
same size → average
different size → larger heap top
```

Complexity

Insertion:

```
text
O(log n)
```

Median:

```
text
O(1)
```

Q79. Task Scheduler

Given tasks like:


```
text
```

```
A A A B B B
```

and cooldown n , determine minimum execution time.

Key idea

The most frequent task determines the required structure.

Formula:

```
text
```

```
(maxFreq - 1) * (n + 1) + numberOfTasksWithMaxFreq
```

Answer:

```
text
```

```
max(formula, tasks.length)
```

Tricky interviewer question

Why do we take the maximum with total task count?

Because if enough other tasks exist, they can fill all idle slots.

Q80. Meeting Rooms

Determine whether one person can attend all meetings.

Sort by start time and compare:

```
text
```

```
current.start < previous.end
```

If true $\rightarrow$ overlap.

Complexity

```
text
```

```
 $O(n \log n)$ 
```

Q81. Meeting Rooms II

Determine minimum number of rooms.

Approach

Min-heap of end times.

For each meeting:

```
text
if earliestEnd <= currentStart
    reuse room
else
    new room
```

Complexity

```
text
O(n log n)
```

Real-world example

Resource allocation for:

```
text
servers
CPU workers
conference rooms
database connections
```

Q82. Jump Game

Determine whether you can reach the last index.

Greedy idea

Maintain furthest reachable position.

```
java
int farthest = 0;

for (int i = 0; i < nums.length; i++) {
    if (i > farthest) return false;

    farthest = Math.max(farthest, i + nums[i]);
}

return true;
```

Complexity

```
text
O(n)
```

SECTION H — GRAPHS

Q83–91

Q83. Number of Islands

Use DFS/BFS over a grid.

When land is found:

```
text
DFS
mark connected land as visited
increment island count
```

Complexity

```
text
O(rows * columns)
```

Tricky question

What if recursion causes stack overflow for a huge grid?

Use iterative BFS/DFS.

Q84. Clone Graph

Use:

```
text
Map<OriginalNode, CloneNode>
```

to avoid duplicating already-visited nodes.

Complexity

```
text
O(V + E)
```

Important

The map also handles cycles.

Q85. Course Schedule

Question:

Can courses be completed given prerequisites?

This is cycle detection in a directed graph.

Approach

Topological sort with indegree.

```
text
indegree = number of prerequisites
queue = zero-indegree nodes
```

If processed count equals number of courses:

```
text
no cycle
```

Complexity

```
text
O(V + E)
```

Q86. Course Schedule II

Instead of only checking feasibility, return the ordering.

Use the same topological-sort algorithm.

Follow-up

What if multiple valid orders exist?

Any valid topological ordering is acceptable unless the problem specifies a deterministic order.

Q87. Rotting Oranges

Classic multi-source BFS.

All initially rotten oranges go into the queue first.

Then process the queue level by level.

Each BFS level represents:

```
text
1 minute
```

Complexity

```
text
O(m*n)
```

Interview insight

Whenever a problem says:

shortest time for all simultaneously spreading sources

think:

multi-source BFS.

Q88. Word Ladder

Find shortest transformation sequence.

Use BFS because every transformation has equal cost.

Key challenge

Generate all valid one-character transformations.

Complexity

Depends on dictionary size and word length.

Interview follow-up

Why BFS rather than DFS?

Because the first time BFS reaches the target it has found the shortest number of edges.

Q89. Network Delay Time

Use Dijkstra when edges have non-negative weights.

Maintain:

```
text
distance[]
PriorityQueue<Node>
```

Complexity

Typical heap implementation:

```
text
 $O((V + E) \log V)$ 
```

Backend example

Finding lowest-latency paths between services/nodes.

Q90. Number of Connected Components

For an undirected graph, use either:

```
text
DFS/BFS
```

or:

```
text
Union-Find
```

Interview follow-up

When is Union-Find especially useful?

When edges are dynamically added and you repeatedly need connectivity checks.

Q91. Redundant Connection

Adding one edge creates a cycle.

Use Union-Find.

For each edge:

```
text
if find(a) == find(b)
    edge is redundant
else
    union(a,b)
```

Complexity

Almost constant amortized per operation with:

```
text
path compression
union by rank/size
```

SECTION I — BACKTRACKING / RECURSION

Q92–95

Q92. Subsets

Given:

```
text
[1, 2, 3]
```

return all subsets.

Pattern

At every element:

```
text
choose it
don't choose it
```

This creates a binary decision tree.

Complexity

```
text
 $O(2^n)$ 
```

because there are 2^n subsets.

Q93. Permutations

For n unique numbers there are:

```
text
 $n!$ 
```

permutations.

Java idea

```
java
void backtrack(
    List<Integer> current,
    boolean[] used
)
```

At each step choose an unused number.

Complexity

```
text
O(n * n!)
```

including output construction.

Q94. Combination Sum

At each candidate:

```
text
choose candidate
continue
or skip
```

The important point is that candidates can be reused in the classic version.

Tricky question

Why don't we move the starting index backward?

Because that would create duplicate combinations in different orders.

Q95. N-Queens

Place one queen per row.

Track occupied:

```
text
columns
```



```
row + col
row - col
```

Complexity

Exponential.

Interview purpose

Tests:

```
text
backtracking
constraint checking
pruning
recursion
state management
```

SECTION J — DYNAMIC PROGRAMMING

Q96–100

Q96. Climbing Stairs

Number of ways to reach step n .

Recurrence:

```
text
dp[n] = dp[n-1] + dp[n-2]
```

Java

```
java
public int climbStairs(int n) {
    int a = 1;
    int b = 1;

    for (int i = 2; i <= n; i++) {
        int next = a + b;
        a = b;
        b = next;
    }
}
```

```
    return b;
}
```

Complexity

```
text
O(n)
O(1)
```

Interview insight

Recognize when a DP problem can be compressed from:

```
text
O(n)
```

space to:

```
text
O(1)
```

because only the previous two states are needed.

Q97. House Robber

Cannot rob adjacent houses.

Recurrence:

```
text
dp[i] = max(
    dp[i-1],
    dp[i-2] + nums[i]
)
```

Java

```
java
public int rob(int[] nums) {
    int prev2 = 0;
    int prev1 = 0;

    for (int money : nums) {
```

```

        int current = Math.max(prev1, prev2 + money);

        prev2 = prev1;
        prev1 = current;
    }

    return prev1;
}

```

Complexity

text

$O(n)$

$O(1)$

Q98. Coin Change

Find minimum number of coins required to make an amount.

DP definition

text

$dp[x]$ = minimum coins needed for amount x

Transition:

text

$dp[x] = \min(dp[x], dp[x - \text{coin}] + 1)$

Complexity

text

$O(\text{amount} * \text{numberOfCoins})$

Tricky question

Why can't greedy always work?

For:

text

$\text{coins} = [1, 3, 4]$

$\text{amount} = 6$

greedy:

text

$4 + 1 + 1 = 3$

optimal:

text

$3 + 3 = 2$

Q99. Longest Increasing Subsequence

Find longest strictly increasing subsequence.

DP solution

text

$dp[i] = \text{LIS ending at } i$

Transition:

text

```
if nums[j] < nums[i]
    dp[i] = max(dp[i], dp[j] + 1)
```

Complexity:

text

$O(n^2)$

Advanced follow-up

Can you do $O(n \log n)$?

Yes, using a `tails` array and binary search.

Important interview progression

A strong interviewer may ask you to go:

text

```
brute force
→ recursion
→ DP  $O(n^2)$ 
→ binary-search optimization  $O(n \log n)$ 
```

Q100. Longest Common Subsequence

Given strings:

```
text
text1
text2
```

find the longest subsequence appearing in both.

DP definition

```
text
dp[i][j]
```

= LCS length for prefixes of `text1` and `text2`.

Transition:

```
text
if characters equal:
    dp[i][j] = 1 + dp[i-1][j-1]

else:
    dp[i][j] = max(
        dp[i-1][j],
        dp[i][j-1]
    )
```

Complexity

```
text
Time: O(m*n)
Space: O(m*n)
```

Space can be optimized to:

```
text
O(min(m,n))
```

Real-world connection

The underlying pattern appears in:

```
text
diff tools
version comparison
```

```
text similarity
sequence comparison
```

The 100-Problem Pattern Map

This is the more important part to memorize.

Pattern	Problems	What interviewer is testing
Hashing	Q1–12	Fast lookup, frequency counting
Prefix Sum	Q12	Transforming range/subarray questions
Sorting + Intervals	Q13–14, Q80–81	Ordering before processing
Two Pointers	Q15, Q29–35	Eliminating nested loops
Sliding Window	Q21–28	Dynamic contiguous ranges
Linked List Pointers	Q36–45	Pointer manipulation
Stack	Q46–55	Nested / next-greater state
Binary Search	Q56–65	Logarithmic search
Trees	Q66–75	DFS/BFS + recursion
Heap	Q76–81	Top-K and scheduling
Greedy	Q79–82	Local decisions with global correctness
Graph BFS/DFS	Q83–88	Traversal/connectivity
Dijkstra/Union-Find	Q89–91	Weighted paths/connectivity
Backtracking	Q92–95	Search + pruning
Dynamic Programming	Q96–100	State + recurrence

What I would expect a 2-year Java candidate to know cold

For your target role, **do not give all 100 problems equal study weight.**

The highest-return problems are:

Tier 1 — Must solve without much help

```
text
Q1    Two Sum
Q4    Group Anagrams
```

Q5 Top K Frequent
Q6 Product Except Self
Q7 Stock
Q8 Maximum Subarray
Q11 Longest Consecutive
Q12 Subarray Sum K
Q13 Merge Intervals
Q22 Longest Substring
Q25 Minimum Window/Sliding Window concepts
Q30 Container With Most Water
Q31 3Sum
Q36 Reverse Linked List
Q37 Cycle Detection
Q39 Merge Lists
Q40 Remove Nth Node
Q46 Valid Parentheses
Q49 Daily Temperatures
Q52 Sliding Window Maximum
Q56 Binary Search
Q58 Rotated Sorted Array
Q62 Binary Search on Answer
Q66 Tree Depth
Q68 Validate BST
Q69 Level Order
Q72 Diameter
Q76 Kth Largest
Q81 Meeting Rooms II
Q82 Jump Game
Q83 Number of Islands
Q85 Course Schedule
Q87 Rotting Oranges
Q96 Climbing Stairs
Q97 House Robber
Q98 Coin Change

Tier 2 — Strong product-company preparation

text

Q17 Find Duplicate
Q18 Set Matrix Zeroes
Q19 Spiral Matrix
Q23 Character Replacement
Q27 Minimum Window
Q33 Trapping Rain Water
Q41 Palindrome Linked List
Q42 Intersection
Q45 Random Pointer
Q47 Min Stack
Q51 Histogram
Q54 Decode String
Q57 First/Last Position
Q59 Rotated Minimum
Q63 Ship Packages
Q65 Median of Two Sorted Arrays
Q70 LCA BST
Q71 LCA Binary Tree
Q74 Kth Smallest BST
Q75 Serialize/Deserialize
Q77 Merge K Lists
Q78 Median Data Stream
Q84 Clone Graph
Q86 Course Schedule II
Q88 Word Ladder
Q89 Network Delay
Q91 Redundant Connection
Q92 Subsets
Q93 Permutations
Q94 Combination Sum
Q95 N-Queens
Q99 LIS
Q100 LCS

What the interviewer is really checking

This is especially important for you because the rest of your preparation is Java/Spring/backend-heavy.

A candidate may solve:

```
text
Two Sum
```

in $O(n)$, but the interviewer can immediately continue:

Why HashMap?

Then:

What happens if there are duplicate keys?

Then:

What is the average lookup complexity?

Then:

What is the worst case?

Then:

How does HashMap handle collisions?

Then:

Why shouldn't your key be mutable?

Then:

What if multiple threads access this map?

Your previous interview-bible material intentionally trained this kind of progression. For example, the earlier Java preparation explicitly emphasized going from `hashCode()` → bucket → collision → `equals()` → resize → mutable-key issue → concurrency rather than stopping at “HashMap uses hashing.”

That same philosophy applies to DSA.

Example of a real interview discussion

Suppose interviewer gives:

Find the longest substring without repeating characters.

A weak answer:

I'll use HashSet.

A stronger 2-year answer:

"I can solve this with a sliding window. I maintain a window [left, right] that always contains unique characters. When the new character is already present, I move left until the duplicate is removed. Every character enters and leaves the set at most once, so the time complexity is $O(n)$ and auxiliary space is $O(\min(n, \text{alphabet size}))$."

Then expect:

Interviewer: Why not brute force?

"That would enumerate substrings and check uniqueness repeatedly, leading to $O(n^2)$ or worse depending on implementation."

Interviewer: Why is the sliding window $O(n)$?

"Although there is a nested while-loop, left only moves forward. Across the entire execution, both pointers move at most $O(n)$ times."

Interviewer: Can you do it without a Set?

"Yes. I can maintain the most recent index of each character in a map and jump left directly to $\text{lastIndex} + 1$."

That is the level at which I recommend you prepare.

Recent 2-year Java/backend interview reports reinforce this style: technical rounds can mix a medium DSA problem with backend fundamentals and project discussion rather than treating DSA as completely isolated. ([LeetCode](#))

The 15 patterns you should actually memorize

Don't memorize 100 solutions independently.

Memorize these **15 patterns**:

text

1. HashMap / HashSet
2. Frequency counting
3. Prefix sum
4. Two pointers
5. Sliding window
6. Fast/slow pointers
7. Monotonic stack
8. Monotonic deque
9. Binary search
10. Binary search on answer
11. DFS
12. BFS

```
13. Heap / PriorityQueue
14. Backtracking
15. Dynamic Programming
```

Once these patterns are understood, many apparently different interview questions become variations of the same underlying problem.

Java-specific DSA expectations for your interviews

Because your target is **Java backend**, don't prepare these problems only in pseudocode.

You should be comfortable writing them using:

```
java
ArrayList
HashMap
HashSet
ArrayDeque
PriorityQueue
Comparator
Arrays.sort()
Collections.sort()
StringBuilder
int[]
long[]
List<int[]>
Map<Integer, Integer>
Map<Character, Integer>
Queue<Node>
Deque<Node>
```

You should also be able to explain:

```
text
ArrayList vs LinkedList
HashMap vs TreeMap
HashSet vs TreeSet
Queue vs Deque
PriorityQueue behavior
Comparable vs Comparator
equals/hashCode
```

```
boxing/unboxing
Integer overflow
recursion stack
mutable vs immutable objects
```

That connects directly with the earlier Java interview preparation, where collection internals, `HashMap`, `equals/hashCode`, `Comparable/Comparator`, iterators and generics were identified as frequent areas for follow-up questions.

Your DSA target for a 2-year Java/Spring Boot role

You do **not** need to approach this as:

“I must solve 100 random LeetCode problems.”

Instead, your target should be:

```
text
Easy
↓
Recognize pattern
↓
Write clean Java
↓
Explain complexity
↓
Handle edge cases
↓
Optimize
↓
Answer interviewer follow-up
↓
Connect to backend scenario
```

Current product-company guidance similarly emphasizes applying fundamentals to solve problems efficiently rather than memorizing isolated topics. ([Amazon.jobs](https://www.amazon.jobs))

Final Module 11 checkpoint

Before moving to Module 12, you should be able to solve these **without looking at the solution**:

```
text
Two Sum
```

```
Longest Substring Without Repeating Characters
Subarray Sum K
Merge Intervals
Maximum Subarray
3Sum
Reverse Linked List
Cycle Detection
Merge Sorted Lists
Valid Parentheses
Daily Temperatures
Binary Search
Rotated Sorted Array
Validate BST
Level Order Traversal
Kth Largest
Meeting Rooms II
Number of Islands
Course Schedule
Rotting Oranges
Climbing Stairs
House Robber
Coin Change
```

And, importantly, for each one you should be able to answer:

“What is the brute-force solution, why is it too slow, what pattern did you recognize, why does the optimized solution work, what is its complexity, what edge cases exist, and how would you implement it cleanly in Java?”

That is the standard I would use for the remainder of this interview bible.